Case Study 4: Locker

Safowan Raiyean · CPEG470 · Writeup and architecture sketch

Architecture sketch

```
Browser
   |  HTTPS (session cookie: locker_sid)
   v
php -S  ->  public/router.php   static file (styles.css) or hand off to index.php
                 |
                 v
        public/index.php        front controller: one if-block per route
                 |              /login /register /logout /forgot /reset /items /items/new /
                 v
        src/bootstrap.php       session start, headers, CSRF helpers, render()
          |-- src/auth.php      users, login/logout, current_user(), login throttle
          |-- src/crypto.php    password hashing, reset tokens
          |-- src/vault.php     vault rows, always filtered by user_id
          |-- src/db.php        PDO + schema + seed  ->  data/locker.db (SQLite)
                 |
                 v
        templates/*.php         HTML, every value printed through e() (htmlspecialchars)

Tables: users(id, email, password_hash, display_name, role)
        vault_items(id, user_id -> users.id, label, secret, notes)
        reset_tokens(email, token_hash, expires_at)      login_attempts(email, at)
```

How the app works

- **Password to SQLite:** `/register` calls `create_user()`, which runs the password through `hash_password()` in `src/crypto.php` and inserts the result into `users.password_hash`. Originally that was a bare `sha256(password)`. Login hashes the typed password the same way and compares.
- **Forgot password:** `POST /forgot` looks up the email and, if it exists, stores a reset token. The page always shows the same message. `/reset` takes email + token + new password, checks the token, and overwrites the hash. No mail is actually sent.
- **Vault scoping:** the session stores only `user_id`. `vault_items_for()` selects `WHERE user_id = ?` with that id, and new rows are inserted with it. There is no route that takes an item id or user id from the request, so there is nothing to tamper with there.
- **Advisor account:** `advisor@campus.edu` is seeded with role `advisor` and holds the org-wide secret (the emergency disbursement PIN, `LOCKER-4410`). Its password is not in the README, so members are not supposed to be able to open that shelf.

The defect: forgeable password-reset token

In `src/crypto.php` the token was `sha1(strtolower(email) . "|" . today's UTC date)`. Nothing in it is secret, so anyone can compute the token for any address. On top of that, `consume_reset_token()` returned `true` when no token was stored at all, so the victim never even had to click "Forgot password". The token was also valid all day and reusable.

How to trigger it (on the unpatched app):

```
1. echo -n "advisor@campus.edu|2026-10-07" | sha1sum        (use today's UTC date)
2. Open /reset, enter advisor@campus.edu, the 40-character hash, and a new password.
3. "Password updated." Sign in as advisor with the new password.
4. The vault shows "Advisor hold" -> LOCKER-4410.
```

I ran this against the original code and got the advisor's secret. The same steps take over any member account.

The patch

- Token is now `bin2hex(random_bytes(32))`, so it cannot be derived from anything.
- Only a SHA-256 of the token is stored, with a 30-minute expiry. It is deleted on use (single use).
- A reset only works if a token was actually issued for that email. The "accept anyway" branch is gone.
- Since there is no mailer, the link is written to the server log as the mail stand-in. It is never returned to the browser.
- A password change kills that user's existing sessions.

Other bugs found and fixed

Bug	Fix
Passwords stored as unsalted SHA-256	`password_hash()` (bcrypt). Old rows are wrapped in bcrypt on first start and replaced at the next login, so nobody is locked out.
Advisor password hardcoded in `db.php` (public repo)	Read from `ADVISOR_PASSWORD` or random. An existing DB with the old password gets it rotated.

No CSRF protection on any form	Per-session token in every POST form, checked in one place in `index.php`.
Session cookie had no flags	HttpOnly, SameSite=Lax, Secure on HTTPS, strict session ids.
Unlimited login guesses	5 failures per account per 15 minutes, then HTTP 429.
Login timing shows which emails exist	Unknown emails do the same bcrypt work.
Old sessions survive a password reset	Session stores a fingerprint of the password hash; mismatch = signed out.
No security headers, PHP version exposed	CSP, X-Frame-Options, nosniff, no-referrer, no-store, HSTS; `X-Powered-By` removed.
PHP errors could print to the page	`display_errors` off, generic 500, details go to the log.
`.edu` check was only "ends with .edu"; no length limits	Must be a valid email ending in .edu; size limits on all fields.
`.gitignore` missed `locker.db-wal/-shm` (WAL mode); Docker copied local DB; data dir 777	Ignore rules added, `.dockerignore` added, data dir 700.
Router served any file found under `public/`	Only known static file types inside `public/`.

Checked after patching

- The forged-token attack returns "token is not valid" and the advisor password is unchanged. The old hardcoded advisor password no longer signs in.
- Normal use still works: demo sign-in, register, store a secret, sign out, and a real reset using the link from the log (works once, second use is rejected).
- A database created by the original app upgrades on first request and the old accounts still sign in.

Known limits (not changed)

- Vault secrets are stored in plain text in SQLite; anyone with the DB file can read them.
- Registration does not prove you own the .edu address, and it tells you if an email is already registered.
- The login pause is per account, so someone can lock a member out for 15 minutes by guessing wrong on purpose.
- It runs on PHP's built-in server, which is meant for development.